Aether: A Domain-Specific Language for Type-Safe LLM Orchestration

A Preprint

Md. Sowad Al-Mughni

May 2026

Abstract

Large language model (LLM) integration in production systems suffers from five systematic engineering failures: runtime-only type checking, complex workflow orchestration without static validation, inadequate testing methodologies, suboptimal caching, and insufficient security guarantees. Existing tools address these problems in isolation: orchestration frameworks provide flexibility without compile-time safety, typed output libraries focus narrowly on schema validation, and security tools operate only at runtime.

This paper presents Aether, a domain-specific language that treats LLM orchestration as a first-class engineering discipline. Aether introduces three core abstractions: `llm fn` for typed LLM interactions, `flow` for DAG-based workflow composition, and `context` for state management. The Aether compiler performs static analysis to verify type contracts across workflow steps, identify parallelization opportunities, and enforce security policies through compile-time taint tracking.

We make five contributions: (1) a type system spanning LLM inputs, outputs, and workflow compositions with compile-time verification; (2) a DAG-based intermediate representation enabling static optimization; (3) a reproducible benchmark methodology for LLM orchestration systems with full JSON artifacts under `bench/results/`; (4) an open-source prototype implementation with OTLP tracing (OpenTelemetry 0.21.0) and Criterion benchmarks; and (5) compile-time taint tracking with measured 100% catch rate on a 60-case adapted InjecAgent corpus, validating that prompt-injection defense can be enforced before any LLM call (Section 9.8, `bench/results/security_v1.json`). Measured outcomes: parallel execution yields a paired BCa speedup of 1.4778× on `customer_support_100` and 2.5841× on `document_analysis_50` (`bench/results/ablation_parallel_v1.json`); the L1 exact-match cache reaches a hit rate of 0.7000 on a 70%-repeat workload and 1.0000 under warm-cache (`bench/results/ablation_cache_v1.json`); the compiler catches 30 of 30 intentionally malformed programs at compile time, vs 17 of 30 caught at runtime by LangChain and DSPy, with 13 missed silently by each baseline (`bench/results/ablation_typesafety_v1.json`). The end-to-end real-OpenAI run (3 trials, both datasets, all three systems) is in `bench/results/real_api_v1.json` (Aether cost $0.128887, $0.478349 total).

Keywords: domain-specific languages, large language models, type systems, workflow orchestration, static analysis

> Reproducibility. Every numeric claim in this paper is traceable to a JSON file in `bench/results/` produced by a specific run. The full inventory:

File	git_version (or library)	Provider	Scope	Trials	measured_at
bench/results/aether_mock_v1.json	4d16ec5cd5cb0957d7dc6408b5df25bacbefbe9b	mock	customer_support_100, document_analysis_50; sequential / parallel / parallel_cached	5	2026-05-03T09:49:57Z
bench/results/langchain_v1.json	9591852f9ebb34822dc628197d47cb643c0ac381 (langchain 0.3.28)	mock	same	5	2026-05-04T05:18:25Z
bench/results/dspy_v1.json	558df452e1e9c36d35bdbc474369862a631c458c (dspy 2.6.27)	mock	same	5	2026-05-08T02:37:56Z
bench/results/aether_real_api_v1.json	9c8001ce269920c98fa733a82b3c69ea7352e37e	openai (gpt-4o-mini)	same	3	2026-05-08T06:29:58Z
bench/results/langchain_real_api_v1.json	langchain 0.3.28	openai	same	3	2026-05-08
bench/results/dspy_real_api_v1.json	dspy 2.6.27	openai	same	3	2026-05-08
bench/results/real_api_v1.json	merged	openai	both datasets, all 3 systems, end-to-end cost	—	2026-05-08T09:26:39Z
bench/results/ablation_cache_v1.json	8aee2cce5f969b5e2d84e9421635535b4dcc0eb7f	mock	customer_support_100, customer_support_repeat_100; no_cache / l1_exact_match / repeat_warm	5	2026-05-08T13:05:22Z
bench/results/ablation_parallel_v1.json	8aee2cce5f969b5e2d84e9421635535b4dcc0eb7f	mock	customer_support_100, document_analysis_50; sequential vs parallel; paired BCa speedup	5	2026-05-08T13:09:48Z
bench/results/ablation_typesafety_v1.json	8aee2cce5f969b5e2d84e9421635535b4dcc0eb7f	n/a (compile-time + mock LLM)	30-case injection corpus across 4 buckets	n/a	2026-05-08T13:15:35Z

File	git_version (or library)	Provider	Scope	Trials	measured_at
bench/results/security_v1.json	2759f1e7f5e26eb4351[illegible]	openai (gpt-4o-mini)	InjecAgent-[illegible]1ea1fc[illegible]: 20 attack + 20 benign cases × 3 trials × 3 configs	3	2026-05-08T18:41:38Z
bench/results/hotpotqa_aether_v1.json	b581653a0611fd[illegible]	[illegible]	[illegible]_500 (latency only; mock LLM does not produce real answers)	3	2026-05-08T10:09:11Z
bench/results/hotpotqa_langchain_v1.json	b581653a0611fd[illegible]	[illegible]	[illegible]	3	2026-05-08T10:12:13Z
bench/results/hotpotqa_dspy_v1.json	b581653a0611fd[illegible]	[illegible]	[illegible]	3	2026-05-08

Reproduction: clone the repo at the listed commit, install `aether-runtime` with the `llm-api` feature for real-API runs, and invoke `scripts/run_benchmark.py` (mock) or `scripts/run_real_api_benchmark.sh` (OpenAI) per Section 13. Markdown summaries of the real-API run and the ablation suite are at `bench/results/REAL_API.md` and `bench/results/ablations_v1.md`.

1. Introduction

Large language models are increasingly deployed in production systems, yet the engineering practices for integrating them remain ad hoc. Developers face fragile prompt chains, unpredictable outputs, absent testing methodologies, and significant operational costs. Existing solutions address these problems piecemeal: orchestration frameworks like LangChain provide flexibility without compile-time safety, while typed output libraries like BAML focus narrowly on schema validation.

Aether is a domain-specific programming language designed to treat LLM orchestration as a first-class engineering discipline. The language introduces three core abstractions: `llm fn` for typed LLM interactions with explicit input/output schemas, `flow` for DAG-based workflow orchestration, and `context` for state management across interactions. The Aether compiler performs static analysis to verify type contracts, identify parallelization opportunities, and generate optimized execution plans.

Current Status: Aether is a working prototype. The compiler (lexer, parser, 5-pass semantic analyzer, DAG code generator) and runtime (parallel execution, LRU caching, multi-provider support) are implemented. The benchmark infrastructure is complete with synthetic datasets and CI integration. All performance claims in this paper are validated against this implementation; Section 9 reports empirical results. Detailed implementation status is in Appendix B.

1.1 Contributions

This paper makes the following contributions:

1. Type System for LLM Orchestration (Section 5): A static type system that spans LLM inputs, outputs, and workflow compositions, enabling compile-time verification of type contracts across workflow steps.
2. DAG-Based Intermediate Representation (Section 6): A compiler architecture that transforms Aether source into a directed acyclic graph representation, enabling static identification of parallelization opportunities and dependency-aware scheduling.

3. Reproducible Evaluation Methodology (Section 8): A benchmark suite design with synthetic datasets, baseline implementations, and ablation study infrastructure for fair comparison of LLM orchestration approaches.
4. Open-Source Prototype (Section 13): A working implementation comprising a compiler (lexer, parser, semantic analyzer, code generator) and runtime (parallel execution, caching, observability), demonstrating feasibility of the approach. The runtime was executed end-to-end against the real OpenAI API (Section 9.6.5) and results are committed in `bench/results/real_api_v1.json`.
5. Compile-Time Taint Tracking with Measured Effectiveness (Sections 9.8, 11): A static taint analysis that distinguishes trusted from untrusted prompt fragments, with a measured 100% compile-time catch rate (CI degenerate) on a 60-case InjecAgent-adapted corpus run against `gpt-4o-mini` (`bench/results/security_v1.json`). Aether blocks the malicious program shape statically, before any LLM call is issued.

2. Problem Statement and Motivation

The integration of LLMs into production software exhibits systematic engineering failures that current tools address incompletely.

2.1 The Type Safety Gap

LLM APIs accept strings and return strings. The semantic structure within those strings (JSON schemas, enumerated values, structured responses) exists only as informal contracts enforced at runtime. When an LLM returns malformed output, the error surfaces far from its source. DSPy (Khattab et al. 2024) introduced typed signatures but remains embedded in Python's dynamic type system. BAML (Boundary ML 2024) generates typed clients but does not extend type checking to workflow composition. Neither provides compile-time verification that a sequence of LLM calls produces type-compatible results.

Measurable problem: In production systems using LangChain, schema validation failures occur at runtime, requiring defensive programming patterns that increase code complexity by an estimated 20-40% (based on manual code review of open-source projects).

2.2 The Orchestration Complexity Problem

Multi-step LLM workflows involve conditional routing, parallel execution, error handling, and state management. LangChain's LCEL provides composability but offers no static guarantees about workflow validity. LangGraph 1.0 (LangChain Inc. 2025) introduced durable state management but relies on runtime graph construction. Temporal (Temporal Technologies 2023) provides robust workflow orchestration but treats LLM calls as opaque activities without domain-specific optimization.

Measurable problem: Workflow errors (unreachable nodes, type mismatches between steps, missing error handlers) are detected only at runtime, extending debugging cycles.

2.3 The Testing Deficit

LLM outputs are probabilistic, making traditional unit testing insufficient. Evaluation frameworks like DeepEval (Confident AI 2024) and LangSmith (LangChain Inc. 2024) provide runtime testing capabilities, but test definitions remain separate from application code. There is no compile-time verification that test assertions match declared output schemas or that evaluation metrics are applicable to specific LLM function signatures.

Measurable problem: Test coverage for LLM applications is typically lower than for traditional software because testing infrastructure is bolted on rather than integrated (Chen et al. 2021).

2.4 The Cost and Latency Challenge

LLM API calls incur latency (hundreds of milliseconds to seconds) and monetary cost. Provider-level prompt caching (Anthropic: 90% cost reduction (Anthropic 2024b); OpenAI: 50% discount (OpenAI 2024)) requires specific prompt structures that are difficult to maintain manually. Semantic caching libraries like GPTCache have seen reduced maintenance, with the project noting it no longer supports new APIs (Zilliz 2024).

Measurable problem: Organizations report 30-60% of LLM API spend could be eliminated with better caching, but implementing caching correctly requires understanding prompt structure at the application level.

2.5 The Security Surface

Prompt injection remains the top risk in OWASP's LLM Top 10 (2025) (OWASP 2025). Runtime guardrails show limited effectiveness: instructional defenses achieve only approximately 70% attack success reduction,

while delimiter isolation provides minimal protection (approximately 85% attack success rate) (Liu et al. 2023). Training-time defenses like StruQ achieve 0% attack success rate (Jatmo et al. 2024), but application-level defenses remain critical for deployed systems.

Measurable problem: No existing framework provides compile-time verification that untrusted input is properly isolated from system prompts across an entire workflow.

2.6 Why a Language-Level Approach

These problems share a common root: LLM integration occurs at runtime, in strings, without static verification. A domain-specific language can address this by moving verification earlier, enabling whole-program analysis, and integrating cross-cutting concerns as language features.

This approach has precedent: SQL moved database queries from string manipulation to a typed query language, enabling query optimization and type checking. Aether aims to do the same for LLM interactions.

2.7 Research Hypotheses

Based on the problems identified above, we formulate three testable hypotheses:

H1 (Type Safety): Compile-time type checking reduces runtime schema and type failures by at least 80% compared to runtime-only validation approaches (LangChain, raw API calls).

H2 (Latency): DAG-based scheduling reduces end-to-end latency by at least 30% on workflows with parallelizable LLM calls compared to sequential execution.

H3 (Cost Efficiency): Compiler-assisted prompt structuring increases cache hit rates by at least 40% compared to manual caching implementations.

These hypotheses correspond to success criteria SC-1, SC-4, and SC-5 respectively. Section 9 presents empirical evaluation of each hypothesis.

3. Related Work

This section surveys the LLM integration landscape as of early 2026, positioning Aether within the existing ecosystem. We acknowledge both the strengths of existing approaches and areas where Aether's design remains unproven.

3.1 Orchestration Frameworks

LangChain/LangGraph (LangChain Inc. 2025): LangChain 1.0 (October 2025) shifted from explicit LCEL pipe operators to a middleware-based architecture. LangGraph 1.0 introduced durable state management enabling agent execution to persist through failures. Strengths include a large ecosystem and production deployment experience. Limitations include runtime-only type checking and no compile-time workflow validation.

DSPy (Khattab et al. 2024): The most conceptually aligned existing approach. DSPy (Stanford, ICLR 2024 Spotlight) treats prompts as declarative signatures with typed inputs and outputs. A compiler automatically optimizes prompts and few-shot examples, demonstrating 25-65% improvement over standard few-shot approaches. Limitation: remains embedded in Python without static cross-module type checking.

LlamaIndex Workflows (LlamaIndex Inc. 2025): Workflows 1.0 (June 2025) provides event-driven, async-first execution with stateful pause/resume. Strong for RAG applications but limited workflow orchestration primitives.

CrewAI (CrewAI Inc. 2025): Distinguishes between Crews (role-based autonomous collaboration) and Flows (deterministic event-driven workflows). Claims 5.76x performance improvement over LangGraph in certain benchmarks, though methodology is not independently verified.

3.2 Typed Output Libraries

BAML (Boundary ML 2024): A domain-specific language providing compile-time type generation for prompts. Key features include 60% fewer tokens than JSON Schema, type-safe streaming, and full LSP tooling. BAML validates the compile-time DSL approach with production adoption. Limitation: focuses on structured outputs without workflow orchestration or caching.

Instructor (Liu 2024): Wraps provider SDKs with Pydantic model integration and automatic retries (3M+ monthly downloads). Runtime-only type enforcement.

Outlines (.txt 2024): Compiles JSON Schema to finite state machines for constrained decoding, achieving 100% structural validity. Runtime compilation, not integrated with workflow orchestration.

Provider APIs: OpenAI and Anthropic offer structured output modes with 100% schema compliance through constrained decoding [Anthropic (2024b)](OpenAI 2024). Limited to single calls without cross-call type verification.

3.3 Evaluation Frameworks

LangSmith (LangChain Inc. 2024): End-to-end observability with tracing, online evaluations, and LLM-as-judge evaluators. Multi-turn evaluation support launched 2025.

DeepEval (Confident AI 2024): Pytest-style unit testing with 50+ built-in metrics including G-Eval and RAG metrics. Version 3.0 added component-level evaluation.

RAGAS (Es et al. 2024): Reference-free RAG evaluation (EACL 2024) with metrics for faithfulness, answer relevancy, and context precision.

3.4 Workflow Engines

Temporal (Temporal Technologies 2023): Provides durable execution through event sourcing, automatic retries, and exactly-once semantics. OpenAI Agents SDK integration announced 2025. Requires manual determinism discipline; no LLM-specific primitives.

Prefect 3.0 (Prefect Technologies 2024): Transactional semantics with 90%+ overhead reduction. ControlFlow framework provides agentic workflows with Pydantic AI integration.

3.5 Security Tools

Guardrails AI (Guardrails AI 2024): Composable validators with configurable on-fail actions. Runtime-only enforcement.

NeMo Guardrails (NVIDIA 2024): Colang DSL for declarative guardrail definitions. Addresses runtime guardrails but not compile-time verification.

StruQ (Jatmo et al. 2024): Training-time defense achieving 0% attack success rate through structured query separation. Demonstrates that architectural approaches outperform runtime guardrails.

3.6 Interoperability Protocols

MCP (Model Context Protocol) (Anthropic 2024a): Anthropic's standard (November 2024) for agent-tool integration using JSON-RPC 2.0. Adopted by OpenAI and Google DeepMind in early 2025.

A2A (Agent-to-Agent Protocol) (Google 2025): Google's standard (April 2025, now Linux Foundation) for agent-to-agent communication. 150+ partner organizations.

3.7 Comparative Analysis

Aspect	LangChain	DSPy	BAML	Temporal	Aether
Abstraction	Library	Compiler	DSL	Workflow Engine	Full DSL
Type Safety	Runtime	Runtime (typed sigs)	Compile-time (output)	None (LLM)	Compile-time (I/O + flow)
Workflow Orchestration	Chain/Graph	Module composition	None	DAG	DAG
Caching	External	None	None	None	Integrated
Evaluation	External (LangSmith)	Built-in metrics	None	None	Language-level (planned)
Security	External	None	None	None	Compile-time taint (planned)
Observability	Built-in	Limited	None	Built-in	Integrated
Durable Execution	LangGraph	None	None	Native	Compilation target (planned)

Honest Assessment: Aether proposes to unify capabilities that exist separately in mature tools. The value proposition depends on whether compile-time verification provides sufficient benefit over runtime approaches to justify learning a new language. This remains to be validated empirically.

4. Design Goals and Measurable Success Criteria

Aether's design is guided by five principles, each with measurable success criteria that will determine whether the language achieves its goals.

4.1 Reliability Through Type Safety

Goal: Catch LLM integration errors at compile time rather than runtime.

Design Approach: - Strong static typing for LLM inputs and outputs with inference across workflow steps - Compile-time verification that LLM function compositions are type-compatible - Runtime fallback with typed error handling for LLM output deviations

Success Criteria (evaluated in Section 9.5): - SC-1: Reduce runtime type errors by >80% compared to equivalent LangChain implementations - SC-2: Achieve <5% false positive rate for compile-time type errors - SC-3: Maintain <10% compile time overhead compared to Python type checking (mypy)

Current Status: Type system implemented for LLM function validation (model/prompt required), symbol resolution, and basic type collection. Full cross-flow type inference in progress.

4.2 Efficiency Through Static Optimization

Goal: Reduce latency and cost through compiler-driven optimization.

Design Approach: - DAG-based intermediate representation enabling parallelization - Compiler-generated caching strategies based on prompt structure analysis - Static identification of batching opportunities

Success Criteria (evaluated in Sections 9.2 and 9.3): - SC-4: Achieve >30% latency reduction on parallelizable workflows compared to sequential execution - SC-5: Achieve >40% cache hit rate improvement through compiler-assisted prompt structuring - SC-6: Reduce API costs by >25% on representative benchmarks through batching and caching

Current Status: Runtime implements level-based parallel execution (JoinSet) and exact-match LRU caching. Benchmark infrastructure is complete with synthetic datasets, Python benchmark runner, provider switching, and CI integration. See Section 8 for methodology and Section 9 for results.

4.3 Modularity Through Composition

Goal: Enable reusable LLM components with clear interfaces.

Design Approach: - `llm fn` as composable units with explicit signatures - `flow` as declarative workflow definitions - `context` as first-class state management

Success Criteria: - SC-7: Achieve >90% code reuse rate for common LLM patterns across projects - SC-8: Reduce lines of code by >30% compared to equivalent Python implementations

Current Status: Syntax fully implemented, parser complete. Semantic analysis tracks `llm fn`, `flow`, `struct`, and `enum` definitions. Success criteria measurable after tooling ecosystem is complete.

4.4 Testability Through Language Integration

Goal: Make LLM testing a first-class language concern.

Design Approach: - Built-in `test` blocks with typed assertions - Golden dataset integration with schema alignment verification - Compile-time validation that test assertions match declared output schemas

Success Criteria: - SC-9: Increase test coverage on LLM applications by >50% compared to baseline - SC-10: Reduce test maintenance burden by >40% through schema-test alignment

Current Status: Test block syntax designed, parser support incomplete.

4.5 Security Through Compile-Time Verification

Goal: Provide security guarantees beyond runtime guardrails.

Design Approach: - Taint tracking distinguishing trusted system prompts from untrusted user input - Compile-time verification of guardrail presence - Static analysis of tool access permissions

Success Criteria: - SC-11: Catch >90% of prompt injection vulnerabilities that pass runtime guardrails in static analysis - SC-12: Zero false negatives for taint tracking violations

Current Status: Security model designed, not yet implemented. Success criteria derived from StruQ research showing architectural approaches outperform runtime guardrails.

5. Language Design

Aether is a statically-typed, domain-specific language for LLM orchestration. This section presents the core abstractions and their semantics. The formal grammar is provided in Appendix A.

5.1 Core Abstractions

5.1.1 LLM Functions (`llm fn`) An `llm fn` encapsulates a single LLM interaction with explicit type contracts:

```
llm fn classify_sentiment(text: string) -> Sentiment {
    model: "gpt-4o",
    temperature: 0.1,
    prompt: "Classify the sentiment of the following text as Positive, Neutral, or Negative.

Text: {{text}}

Respond with only the sentiment classification."
}

enum Sentiment { Positive, Neutral, Negative }
```

Semantics: - Input parameters are type-checked at compile time - The output type constrains the expected LLM response - The compiler generates parsing and validation code for the output type - Runtime `ParseError` is raised if the LLM output does not conform to the schema

5.1.2 Flows (`flow`) A `flow` defines an orchestrated workflow as a directed acyclic graph:

```
flow analyze_document(doc: string) -> AnalysisResult {
    // These calls can execute in parallel (no data dependency)
    let sentiment = classify_sentiment(text: doc);
    let entities = extract_entities(document: doc);

    // This call depends on sentiment result
    let action = if sentiment == Sentiment.Negative {
        determine_action(text: doc, urgency: "high")
    } else {
        determine_action(text: doc, urgency: "normal")
    };

    return AnalysisResult {
        sentiment: sentiment,
        entities: entities,
        recommended_action: action
    };
}
```

Semantics: - The compiler constructs a dependency graph from data flow - Independent calls are candidates for parallel execution - Type checking verifies that all branches return compatible types

5.1.3 Contexts (`context`) A `context` defines managed state across interactions:

```
context ConversationState {
    history: list<Message>,
    user_preferences: UserPrefs,
    session_id: string
}

struct Message {
    role: string,
    content: string,
    timestamp: int
}
```

Semantics: - Contexts are serializable and can be persisted across runtime boundaries - The compiler generates serialization/deserialization code - Context access within flows is tracked for state management optimization

5.2 Type System

Aether's type system includes:

Primitive Types: `string`, `int`, `float`, `bool`

Structured Types: `struct` definitions with named fields

Enumerated Types: `enum` definitions for categorical outputs

Collection Types: `list<T>`, `map<K, V>`, `optional<T>`

Constrained Types (planned): Refinement types for values with constraints

```
type Rating = int where 1 <= value <= 5
type NonEmptyString = string where length > 0
```

5.3 Error Handling

Aether provides structured error handling for LLM-specific failures:

```
flow robust_classification(text: string) -> Sentiment {
    try {
        return classify_sentiment(text: text);
    } catch (e: ParseError) {
        // LLM output did not match expected schema
        log("Parse failed: " + e.message);
        fallback {
            return classify_sentiment_simple(text: text);
        }
    } catch (e: RateLimitError) {
        // Provider rate limit exceeded
        retry with exponential_backoff(max_attempts: 3);
    } catch (e: ModelError) {
        // Model unavailable or failed
        fallback {
            return Sentiment.Neutral;  // Safe default
        }
    }
}
```

5.4 Testing Constructs

Tests are first-class language constructs:

```
test "sentiment_classification_accuracy" {
    let positive_texts = golden_dataset("sentiment/positive.jsonl");

    for text in positive_texts {
        let result = classify_sentiment(text: text.input);
        assert result == Sentiment.Positive
            with tolerance: 0.95  // Allow 5% error rate
            with metric: accuracy;
    }
}

test "entity_extraction_completeness" {
    let test_doc = "John Smith works at Acme Corp in New York.";
    let result = extract_entities(document: test_doc);

    assert result.persons.contains("John Smith");
    assert result.organizations.contains("Acme Corp");
    assert result.locations.contains("New York");
}
```

6. Compiler Architecture

This section describes the Aether compiler pipeline. All compiler phases (lexer through code generator) are fully implemented. See Appendix B for detailed component status.

6.1 Compiler Pipeline

6.1.1 Lexical Analysis The lexer converts Aether source code into tokens. It supports:

- All Aether keywords (`llm`, `fn`, `flow`, `context`, `test`, `struct`, `enum`, etc.)
- Operators and delimiters
- String literals with template interpolation (`{{variable}}`)
- Comments (single-line `//` and multi-line `/* */`)

The lexer is implemented in Rust using the logos crate with comprehensive test coverage.

6.1.2 Syntactic Analysis The parser constructs an Abstract Syntax Tree from tokens. Full support includes:

- Type declarations (`struct`, `enum` with associated data like `Variant(Type)`)
- Complete `llm fn` declarations with model, temperature, prompt, system prompt
- Full `flow` definitions with control flow (`if`/`else`, `match`, `for`, `while`)
- String templates with `{{variable}}` interpolation
- Context definitions
- Basic `test` block structure

The parser is implemented as a hand-written recursive descent parser (approximately 1900 lines) with comprehensive error recovery and span tracking.

6.1.3 Semantic Analysis The semantic analyzer implements a comprehensive 5-pass analysis:

Pass 1 - Symbol Collection: Gather all type definitions (`struct`, `enum`, `type` aliases) and function signatures (`llm fn`, `flow`, `fn`) into a hierarchical symbol table with scope management.

Pass 2 - Type Internals Validation: Detect duplicate fields in structs, duplicate variants in enums, and validate internal consistency.

Pass 3 - LLM Function Validation: Verify that `model` and `prompt` are present, validate template references (`{{param}}`, `{{context.KEY}}`, `{{node.ID.output}}`), check parameter usage.

Pass 4 - Flow Analysis: Forward-only type inference for expressions (literals, identifiers, function calls, field access, struct literals, enum variants), call validation (argument count, type compatibility), return type verification.

Pass 5 - Function Analysis: Validate regular function bodies with the same type inference and return type checking.

Error Infrastructure:

- Source locations (line, column) in all error messages
- Error accumulation (collects up to 10 errors before aborting)
- "Did you mean?" suggestions using Levenshtein distance for undefined symbols
- 15+ distinct error types including `UndefinedSymbol`, `TypeMismatch`, `CircularDependency`

6.1.4 Intermediate Representation The IR is a DAG JSON format where:

- Nodes represent operations (`LlmFn`, `Compute`, `Conditional`, `Input`, `Output` types)
- Dependencies are computed from data flow (variable bindings to prior node outputs)
- Cycle detection via Kahn's algorithm topological sort, rejecting flows with circular dependencies
- Template refs provide machine-readable metadata for each `{{placeholder}}`:
  - `kind`: `context`, `node_output`, `parameter`, `constant`, `variable`
  - `path`, `node_id`, `field`, `required`, `sensitivity`
- Execution hints support future scheduling: `parallel_group`, `max_concurrency`, `error_policy`
- Render policy enables security enforcement: `allowed_context_keys`, `redact_keys`, `escape_html`

6.2 Planned Optimizations

The optimizer will implement the following transformations (not yet implemented):

Parallelization: Independent LLM calls within a flow will be identified and scheduled for parallel execution based on dependency analysis.

Caching Strategy Generation: The compiler will analyze prompt structures to generate caching strategies (exact-match, prefix caching, semantic caching).

Common Subexpression Elimination: Identical LLM calls with the same inputs will be executed once and results reused.

6.3 Code Generation Targets (Planned)

The code generator will support multiple backends:

- Python: Primary target for integration with existing ML ecosystems
- Rust: High-performance native execution
- Temporal workflows: Durable execution for long-running agents
- WebAssembly: Browser and edge deployment

7. Runtime Architecture

The Aether runtime executes compiled workflows and manages caching, context, and observability. The runtime MVP is implemented with core functionality operational.

7.1 Execution Engine

The execution engine provides:

- Topological sorting with cycle detection using petgraph for correct execution order
- Level-based parallel execution grouping independent nodes and executing them concurrently via tokio JoinSet
- Sequential execution mode via `?sequential=true` query parameter forcing `max_concurrency=1` for ablation studies
- Dependency-aware output access with `{{node.ID.output}}` template substitution from upstream results
- Error policies (Fail, Skip, Retry) controlling execution behavior on node failure
- Node status tracking with states: Pending, Running, Succeeded, Failed, Skipped
- Latency percentile computation (p50/p95/p99) for both node execution times and level execution times, computed server-side

7.2 Caching Layer

A multi-level caching system:

Level 1: Exact-Match Cache (Implemented)

- Key: SHA256 hash of (model + rendered_prompt + temperature + max_tokens)
- Storage: In-memory LRU cache with configurable size (default 1000 entries)
- Cache hits return stored response with 0 token cost, flagged as `cached: true`
- `tokens_saved` tracking for cumulative savings metrics

Level 2: Semantic Cache (Planned)

- Key: Embedding vector of the prompt
- Storage: Vector database
- Hit condition: Cosine similarity above configurable threshold

Level 3: Provider Prefix Cache (Planned)

- Leverage Anthropic and OpenAI prompt caching
- Compiler generates prompts with stable prefixes

7.3 Context Management

The context manager provides:

- ContextStore trait abstraction for pluggable persistence backends
- InMemoryContextStore implementation (MVP) with RwLock for thread-safety
- ExecutionContext struct with variables (HashMap), metadata, execution_id
- Nested path access via `get_path(&["user", "profile", "name"])` for deep value retrieval
- Future backends planned: RedisContextStore, FileContextStore, PostgresContextStore

7.4 Observability

Built-in observability includes:

- Structured logging: tracing crate with spans for all LLM interactions
- Distributed tracing: OpenTelemetry 0.21.0 with OTLP export (replaces deprecated Jaeger exporter)
  - Compatible with Jaeger, Zipkin, and other OTLP-compliant backends
  - `tracing-opentelemetry` 0.22.0 layer integration
  - Configurable via `JAEGER_COLLECTOR_ENDPOINT` or `JAEGER_AGENT_ENDPOINT` environment variables
- Metrics export: Prometheus-compatible `/metrics` endpoint
- Per-execution response fields: `level_execution_times_ms`, `node_execution_times_ms`, `tokens_saved`
- Criterion benchmarks: Native Rust benchmarks in `aether-runtime/benches/` for DAG execution profiling

7.5 Template Engine

Prompt template rendering with:

- {{context.KEY}} substitution from ExecutionContext variables
- {{node.ID.output}} substitution from upstream node outputs
- Nested path access for deep context values
- TemplateRef metadata from compiler for validation
- Sensitivity tracking with optional redaction
- Deterministic rendering for cache key stability

7.6 LLM Provider Interface

Provider abstraction with:

- LlmClient trait: `async fn complete(request) -> Result<LlmResponse>`
- MockLlmClient: Deterministic responses, configurable latency, failure simulation
- OpenAIClient: Real API integration (behind feature flag)
- AnthropicClient: Real API integration (behind feature flag)
- AETHER_PROVIDER environment variable: Force provider selection for benchmarking (`mock|openai|anthropic`)

8. Evaluation Methodology

This section describes the evaluation methodology for validating Aether's claimed benefits. Results from executing this methodology appear in Section 9.

8.1 Implementation Status

Full benchmark infrastructure is implemented. The runtime and tooling provide complete measurement capabilities for reproducible benchmarking:

- Synthetic Datasets (`bench/datasets/`): CustomerSupport-100 and DocumentAnalysis-50
- Benchmark Runner (`scripts/run_benchmark.py`): Cold/warm/sequential modes, latency percentiles, JSON output
- Provider Switching (`AETHER_PROVIDER`): Environment variable for deterministic CI benchmarks
- CI Integration (`.github/workflows/benchmark.yml`): Automated runs with PR comments and artifact upload

8.2 Benchmark Suite Design

8.2.1 Datasets

Dataset	Task	Size	Status
CustomerSupport-100	Urgency/category triage	100 queries	Implemented
DocumentAnalysis-50	Parallel entity/summary extraction	50 documents	Implemented
CustomerSupport-1K	Extended triage benchmark	1,000 queries	Planned
RAG-QA-1K	Retrieval + generation	1,000 questions	Planned

CustomerSupport-100 Schema:

```
{
  "id": "cs_001",
  "query": "Customer query text...",
  "expected_urgency": "Low|Medium|High|Critical",
  "expected_category": "billing|technical_support|...",
  "context": {
    "customer_tier": "free|basic|premium|enterprise",
    "previous_tickets": 0
  }
}
```

8.2.2 Metrics Efficiency Metrics:

- End-to-end latency (p50, p95, p99)
- API cost per 1,000 requests
- Cache hit rate (exact-match)
- Parallelization factor (concurrent calls / sequential calls)

Quality Metrics:

- Schema conformance rate
- Error rate by category (parse, rate limit, model)

Developer Experience Metrics:

- Lines of code (Aether vs. Python baseline)
- Compile-time error detection rate

8.3 Baseline Comparisons

Each benchmark compares:

1. Python + LangChain: Simulates sequential execution, manual caching (15% hit rate)
2. Python + DSPy: Module-based composition, no caching
3. Aether: DAG-based parallel execution with integrated caching

Baseline stubs are implemented in `bench/baselines/` with mock mode support.

8.4 Ablation Studies

To isolate the contribution of each feature:

1. Caching ablation: Compare (a) no caching, (b) exact-match only, (c) full multi-level caching
2. Parallelization ablation: Compare sequential vs. parallel execution using `?sequential=true`
3. Type safety ablation: Measure errors caught at compile time vs. runtime

8.5 Statistical Methodology

Every aggregate cell in Section 9 follows the convention `mean ± std [95% CI]`, where the CI is computed by bias-corrected and accelerated (BCa) bootstrap with `n_resamples=10000` and `seed=42`, falling back to the percentile method when per-trial variance is degenerate. The implementation lives in `_bootstrap_ci` at `scripts/run_benchmark.py:546-593` and is invoked uniformly by all benchmark drivers.

Trial counts. The trial budget per (system, dataset, config) is recorded in the `trials_per_config` field of every JSON in `bench/results/`:

- Mock-mode runs (`aether_mock_v1.json`, `langchain_v1.json`, `dspy_v1.json`) and the three ablations (`ablation_cache_v1.json`, `ablation_parallel_v1.json`, `ablation_typesafety_v1.json`): 5 trials.
- Real-API runs (`aether_real_api_v1.json`, `langchain_real_api_v1.json`, `dspy_real_api_v1.json`) and the security suite (`security_v1.json`) and the HotpotQA suite (`hotpotqa_*_v1.json`): 3 trials, capped to control OpenAI cost (the real-API run hit `actual_cost_usd=0.478349` against a `budget_usd=10.0` gate).

Speedup ratios. Speedup is computed by paired-trial bootstrap on the ratio `sequential.p50 / parallel.p50`, using the same BCa parameters. The definition string is recorded inside the speedup field of each `ablation_parallel_v1.json` `results[*].speedup` block (e.g. `results[@ref2].speedup.definition`), so the metric definition travels with the data.

Cross-mode deltas. Cache and parallelization comparisons across modes use paired-trial deltas (e.g. `latency_p50_delta_warm_vs_no_cache`, recorded in `ablation_cache_v1.json` `cross_mode_deltas`), again with the same BCa parameters.

Significance testing. Pairwise differences are reported as overlapping/non-overlapping 95% CIs rather than as p-values from formal hypothesis tests. The reader can read significance off the CIs directly; we did not run separate Welch's t-tests or Mann–Whitney U tests. This is a deliberate scope cut: the trial counts are small (3–5) and the load-bearing comparisons (e.g. parallel speedup) have CIs that do not overlap 1.0 by margins of multiple standard deviations.

Mock-mode determinism. The mock LLM provider returns a fixed-latency placeholder (50 ms flat per call, per `bench/results/ablations_v1.md` methodology footer). Trial-to-trial variance in mock runs therefore reflects scheduler and HTTP-loopback jitter only, not model behavior. This is why mock-mode standard deviations are sub-millisecond on most cells.

Hardware uniformity. All ablation files (`ablation_*.json`) and `aether_mock_v1.json` were measured on `Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz`, RAM 7.71 GiB, Ubuntu 24.04.4 LTS (recorded in each JSON's `hardware` block). The real-API runs were measured on the same Linux host. The `langchain_v1.json`, `dspy_v1.json`, and `hotpotqa_*_v1.json` mock runs were measured on Windows-11 against the same physical CPU model; cross-OS variance is discussed in Section 9.7.4.

9. Evaluation Results

This section presents empirical results from executing the benchmark suite described in Section 8. Mock-mode measurements use a fixed-latency mock LLM provider (50 ms flat per call) to isolate runtime/orchestration jitter from model variance; real-API measurements against `gpt-4o-mini` are reported in Section 9.6.5 and the new HotpotQA / security subsections (9.8, 9.9). Every numeric cell below is anchored to a JSON file in `bench/results/`; the field path is given alongside each table.

9.1 Summary of Results

Hypothesis	Target	Measured Result	Source
H1 (Type Safety)	>80% reduction in runtime type errors	30/30 (100.00%) caught at compile time; LangChain 17/30 at runtime + 13 missed silently; DSPy 17/30 at runtime + 13 missed silently	`ablation_typesafety_v1.json` `summary.aether_caught` / `summary.lc_*` / `summary.dspy_*`

Hypothesis	Target	Measured Result	Source
H2 (Latency, customer_support_100)	>30% reduction via parallelization	speedup = 1.4778× [1.4728, 1.4849] (paired BCa)	`ablation_parallel_v1.json results[@ref2].speedup.speedup_p50`
H2 (Latency, document_analysis_50)	>30% reduction via parallelization	speedup = 2.5841× [2.5635, 2.5986] (paired BCa)	`ablation_parallel_v1.json results[@ref5].speedup.speedup_p50`
H3 (Cost Efficiency, repeat workload)	>40% improvement in cache hit rate	0.7000 (l1_exact_match) and 1.0000 (repeat_warm) on `customer_support_repeat_100`, both CIs degenerate (constant per-trial)	`ablation_cache_v1.json results[@ref5].cache_hit_rate.mean`, `results[@ref6].cache_hit_rate.mean`
H4 (Security, taint tracking)	(new — see Section 9.9)	compile_time_catch_rate = 1.0000 on `aether_taint_on`, 0.0000 on `aether_taint_off` and `langchain_baseline`; ASR is 0.0 across all configs (model-side rejection on `gpt-4o-mini`)	`security_v1.json configs[*].metrics`

9.2 Latency Analysis (H2)

The parallelization ablation runs `customer_support_100` and `document_analysis_50` against the runtime in two modes — `sequential` (POSTed with `?sequential=true`) and `parallel` (default) — clearing the cache before every trial in both modes so the parallelization signal is not confounded with caching. Speedup is the paired-trial BCa bootstrap of `sequential.p50 / parallel.p50`. Cells are `mean ± std [95% CI]`.

customer_support_100 (`ablation_parallel_v1.json results[@ref0]`, `results[@ref1]`, `results[@ref2]`):

Configuration	p50 (ms)	p95 (ms)	p99 (ms)	parallelization_factor	speedup_p50
Sequential	206.0 ± 0.71 [205.4, 206.6]	213.87 ± 1.89 [212.09, 215.06]	246.70 ± 30.78 [224.96, 273.04]	1.0 ± 0.0 [1.0, 1.0]	1.0 (baseline)
Parallel	139.4 ± 0.55 [139.0, 139.8]	148.01 ± 1.88 [146.6, 149.41]	155.34 ± 6.08 [151.13, 161.17]	1.47 ± 0.00 [1.47, 1.48]	1.48 [1.47, 1.48]
Parallel + cache (warm)	33.0 ± 0.71 [32.4, 33.6]	39.02 ± 1.24 [37.60, 39.65]	40.65 ± 1.60 [39.61, 42.33]	1.3724 (raw_trial_results mean)	derived from above

The "Parallel + cache (warm)" row is sourced from `aether_mock_v1.json results[@ref2]` (`customer_support_100`, `parallel_cached`, 5 trials). Cache hit rate for that row is `1.0000` (CI degenerate), `cache_hits=300, cache_misses=0` per trial.

document_analysis_50 (`ablation_parallel_v1.json results[@ref3]`, `results[@ref4]`, `results[@ref5]`; `aether_mock_v1.json results[@ref5]`):

Configuration	p50 (ms)	p95 (ms)	p99 (ms)	parallelization_factor	speedup_p50
Sequential	218.6 ± 0.55 [218.2, 219.0]	224.66 ± 1.57 [223.42, 225.76]	232.82 ± 6.36 [228.13, 237.62]	1.0 ± 0.0 [1.0, 1.0]	1.0 (baseline)
Parallel	84.6 ± 0.65 [84.1, 85.2]	90.95 ± 3.55 [88.86, 94.75]	100.12 ± 8.83 [93.726, 107.58]	2.56 ± 0.01 [2.55, 2.56]	2.58 [2.56, 2.60]

Configuration	p50 (ms)	p95 (ms)	p99 (ms)	parallelization_factor	speedup_p50
Parallel + cache (warm)	31.5 ± 0.71 [31.0, 32.1]	38.71 ± 0.44 [38.2, 39.0]	39.92 ± 1.05 [39.18, 40.73]	1.8724 (raw_trial_results mean)	derived from above

Methodology. 5 trials per cell. Mock LLM at 50 ms flat. `parallelization_factor` is the runtime response field of the same name (`sum(node_execution_times_ms) / total_execution_time_ms`), read directly from each trial response. Speedup CIs are paired-trial BCa bootstrap (n_resamples=10000, seed=42); the definition string is recorded in `ablation_parallel_v1.json results[@ref2].speedup.definition` and `results[@ref5].speedup.definition`.

Discussion. Speedup tracks workflow shape: the document-analysis DAG has a wider parallel level ($ $3 concurrent LLM calls per item, parallelization_factor ≈ 2.56) and yields a 2.58× p50 speedup; the customer-support DAG has a narrower one ($ $1.5 concurrent calls, parallelization_factor ≈ 1.47) and yields a 1.48× p50 speedup. The earlier paper's "2.7× speedup" extrapolation assumed perfectly parallel workloads; the measured value depends on the DAG.

9.3 Caching Performance (H3)

Cache hit rate, p50 latency, and warm-vs-no-cache delta across three cache modes (`no_cache` clears the cache before every individual `/execute`; `l1_exact_match` clears once per trial; `repeat_warm` runs the dataset once as warmup, discards the warmup latencies, then measures a second pass over the populated cache). 5 trials per cell, mock LLM at 50 ms flat. Source: `ablation_cache_v1.json`. Cells are `mean ± std [95% CI]`.

`customer_support_100` — every query is unique (`results[@ref0]`, `results[@ref1]`, `results[@ref2]`, `results[@ref3].cross_mode_deltas`):

Mode	p50 (ms)	cache_hit_rate	Δ p50 vs no_cache (ms)
no_cache	144.8 ± 0.45 [144.2, 145.0]	0.0000 (CI degenerate)	—
l1_exact_match	143.8 ± 1.52 [142.6, 144.9]	0.0000 (CI degenerate)	(cache empty within run)
repeat_warm	33.2 ± 0.84 [32.4, 33.8]	1.0000 (CI degenerate)	-111.6 [-112.6, -111.0]

`customer_support_repeat_100` — 30 unique queries × repeats (`results[@ref4]`, `results[@ref5]`, `results[@ref6]`, `results[@ref7].cross_mode_deltas`):

Mode	p50 (ms)	cache_hit_rate	Δ p50 vs no_cache (ms)
no_cache	145.1 ± 0.22 [145.0, 145.4]	0.0000 (CI degenerate)	—
l1_exact_match	36.1 ± 1.43 [34.9, 37.2]	0.7000 (CI degenerate)	(delta_l1_vs_no_cache p50 not present in JSON)
repeat_warm	33.8 ± 1.92 [32.8, 36.0]	1.0000 (CI degenerate)	-111.3 [-112.3, -108.8]

Methodology. Hit rate is read directly from the runtime's `cache_hit_rate` response field, not derived. `tokens_saved_total` is reported in every JSON cell as 0.0 across every config because the mock LLM does not populate the `tokens_saved` field of the response; we did not measure mock-mode token savings or convert them to a dollar figure. The earlier paper's "60% hit rate", "18,240 tokens saved" and "$0.91 cost reduction" cells came from an extrapolation that no committed JSON in `bench/results/` supports — those cells are therefore dropped. Real-API token usage and cost are reported in Section 9.6.5 above (Aether $0.128887 across 3 trials × 2 datasets).

Discussion. When every query is unique within a single run (`customer_support_100`), `l1_exact_match` cannot help (hit rate stays at 0.0); only the cross-run `repeat_warm` mode achieves a hit. When the workload contains repeats (`customer_support_repeat_100`, designed so 70 of 100 queries are repeats of earlier ones), `l1_exact_match` reaches 0.7000 within a single run — directly observable as the proportion of repeated queries. `repeat_warm` reaches 1.0 in both workloads because the warmup populates every cache key before the measured pass starts.

9.4 Code Complexity Comparison

We did not measure equivalent-functionality LOC for this paper revision. An earlier draft reported 253/287 (LangChain), 283/312 (DSPy), 78/111 (Aether) lines for the CustomerSupport-Triage and DocumentAnalysis-Pipeline case studies, but those numbers are not anchored to any JSON file in `bench/results/` and the Aether source artifacts they were extracted from (standalone `customer_support_triage.aether` and `document_analysis_pipeline.aether` programs equivalent to the LangChain triage/extraction blocks at `bench/baselines/langchain_baseline.py:289-376` and DSPy equivalents) are not committed in the repository at the commits listed in the Reproducibility callout.

A partial LOC comparison is possible: the LangChain triage chain (`build_triage_chain`, `bench/baselines/langchain_baseline.py:299-329`) and extraction chain (`build_extraction_chain`, `bench/baselines/langchain_baseline.py:355-376`) are the case-study sources for the Python side. Counting the Aether side requires checking in the equivalent `.aether` programs first. We deferred that step to a follow-up paper revision rather than report unbacked numbers in the current one.

Reading guidance: the qualitative code-complexity contrast — Aether's `flow` keyword and `llm fn` declaration vs LangChain's chain-of-LCEL-pipes or DSPy's `Module`/`Predict` boilerplate — is documented in Section 6 (the case-study code listings at lines 762-829 are themselves a small visual benchmark). LOC numbers will return in a future revision once the matched `.aether` sources land in `bench/datasets/` or `examples/`.

9.5 Type Safety Analysis (H1)

Error detection comparison on a 30-case corpus of intentionally malformed programs split across four buckets. For each case, an `aetherc check` is run on the `.aether` source and a `python` is run on the LangChain and DSPy equivalents; results are classified by stderr pattern (Aether) or by exit code + traceback class (Python). Source: `bench/results/ablation_typesafety_v1.json`, `summary.by_bucket` and `summary` blocks. Companion Markdown: `bench/results/ablations_v1.md`.

Bucket	Total	Aether (compile-time)	LangChain (runtime)	LangChain (missed silently)	DSPy (runtime)	DSPy (missed silently)
`type_mismatch`	10	10	0	10	0	10
`undefined_reference`	10	10	10	0	10	0
`missing_field`	5	5	5	0	5	0
`duplicate_definition`	5	5	2	3	2	3
Total	30	30	17	13	17	13

Source per row: `summary.by_bucket.<bucket>.{total, aether_caught, lc_caught_at_runtime, dspy_caught_at_runtime}`. Aether's per-bucket missed-silently count is 0 in all buckets (`summary.aether_missed = 0`); LangChain and DSPy missed-silently counts are computed as `total - caught_at_runtime` per bucket and cross-checked against `summary.lc_missed_silently=13`, `summary.dspy_missed_silently=13`.

Compile-time detection rate: 30/30 = 100.00% for Aether (`summary.aether_caught=30`, `summary.aether_missed=0`). The 13 cases LangChain/DSPy miss silently exit code 0 with a plausible-looking output — the most dangerous failure mode in production, because it produces output that looks valid.

Methodology. `aetherc check <file>` exit-coded by stderr pattern: exit 0 → missed; exit ≠ 0 with a known `SemanticError` variant pattern → `caught_at_compile_time`. `python <file>` with a 30 s timeout: exit 0 → `missed_silently`; exit ≠ 0 with a Python traceback → `caught_at_runtime` (this includes `SyntaxError` at file load and Enum class-body errors). The `error_class_matched` and `exception_class` fields are recorded per case in `test_cases[*]` and reproduced in `bench/results/ablations_v1.md`.

Methodology note (cd → dd substitution). Verbatim from *`ablation_typesafety_v1.json methodology_notes.cd_substitution`*:

> The original ablation design included a `circular_dependency` category, but verification revealed that aetherc's source-level cd detector is currently preempted by semantic analysis

on programs that contain other issues; the `SemanticError::CircularDependency` variant is defined but never emitted. Rather than fabricate test cases that would not trigger the intended error path, we substituted `duplicate_definition` tests, which exercise a different but more practically significant error class (silent shadowing in Python is more dangerous than a circular dependency, which typically manifests as `RecursionError` or `ImportError` — loud and visible). The cd detection gap is tracked at https://github.com/sowadalmughni/aether-lang/issues/4 and is targeted for a follow-up compiler release.

9.6 Case Study: Customer Support Triage

This section presents an end-to-end case study demonstrating Aether's capabilities on a realistic customer support workflow.

9.6.1 Workflow Description The triage workflow: 1. Classifies customer query urgency (Low, Medium, High, Critical) 2. Categorizes the query type (billing, technical_support, account, general) 3. Generates an initial response draft 4. Determines routing (human escalation vs. automated response)

9.6.2 Aether Implementation

```
enum Urgency { Low, Medium, High, Critical }
enum Category { Billing, TechnicalSupport, Account, General }

struct TriageResult {
    urgency: Urgency,
    category: Category,
    response_draft: string,
    escalate: bool
}

llm fn classify_urgency(query: string, customer_tier: string) -> Urgency {
    model: "gpt-4o",
    temperature: 0.1,
    prompt: "Classify the urgency of this customer support query.

Customer Tier: {{customer_tier}}
Query: {{query}}

Respond with exactly one of: Low, Medium, High, Critical"
}

llm fn classify_category(query: string) -> Category {
    model: "gpt-4o",
    temperature: 0.1,
    prompt: "Classify the category of this customer support query.

Query: {{query}}

Respond with exactly one of: Billing, TechnicalSupport, Account, General"
}

llm fn draft_response(query: string, urgency: string, category: string) -> string {
    model: "gpt-4o",
    temperature: 0.7,
    prompt: "Draft a professional response to this customer query.

Urgency: {{urgency}}
Category: {{category}}
Query: {{query}}

Provide a helpful response."
}

flow triage_customer_query(query: string, customer_tier: string) -> TriageResult {
    // These execute in parallel (no data dependency)
    let urgency = classify_urgency(query: query, customer_tier: customer_tier);
    let category = classify_category(query: query);
```

```
    // This depends on urgency and category
    let response = draft_response(
        query: query,
        urgency: to_string(urgency),
        category: to_string(category)
    );

    let escalate = urgency == Urgency.Critical ||
                   (urgency == Urgency.High && customer_tier == "enterprise");

    return TriageResult {
        urgency: urgency,
        category: category,
        response_draft: response,
        escalate: escalate
    };
}
```

9.6.3 Compiled DAG (Excerpt)

```
{
  "version": "1.0",
  "name": "triage_customer_query",
  "nodes": [
    {
      "id": "input",
      "type": "Input",
      "outputs": {"query": "string", "customer_tier": "string"}
    },
    {
      "id": "classify_urgency_1",
      "type": "LlmFn",
      "depends_on": ["input"],
      "execution_hints": {"parallel_group": 0}
    },
    {
      "id": "classify_category_1",
      "type": "LlmFn",
      "depends_on": ["input"],
      "execution_hints": {"parallel_group": 0}
    },
    {
      "id": "draft_response_1",
      "type": "LlmFn",
      "depends_on": ["classify_urgency_1", "classify_category_1"],
      "execution_hints": {"parallel_group": 1}
    }
  ]
}
```

The compiler identifies that `classify_urgency_1` and `classify_category_1` can execute in parallel (same `parallel_group`), while `draft_response_1` must wait for both.

9.6.4 Compile-Time Errors Caught The following errors are caught at compile time in this workflow:

1. Type mismatch: If `draft_response` declared `urgency: int` instead of `urgency: string`, compiler error at line N
2. Undefined reference: If `classify_urgeny` (typo) called, compiler suggests "Did you mean 'classify_urgency'?"
3. Missing required field: If `TriageResult` return missing `escalate` field, compiler error
4. Enum variant mismatch: If `urgency == Urgency.Urgent` (invalid variant), compiler error listing valid variants

9.6.5 Performance Results (real OpenAI API) The case study was executed end-to-end against the real `gpt-4o-mini` API on 2026-05-08 (3 trials, 100 items, all three configurations); the orchestration script was `scripts`

/run_real_api_benchmark.sh and the merged result is bench/results/real_api_v1.json, with a Markdown summary at bench/results/REAL_API.md. Cells are mean ± std [95% CI] across the 3 trials.

Metric	Sequential	Parallel	Parallel + Cache (warm)
Aether p50 (ms)	6821.0 ± 636.32 [6121.5, 7235.67]	5395.17 ± 69.21 [5354.17, 5475.0]	37.33 ± 2.08 [35.0, 38.67]
Aether p95 (ms)	11609.82 ± 2528.15 [8859.45, 13267.48]	8148.40 ± 648.04 [7427.35, 8566.68]	41.0 ± 0.0 [41.0, 41.0]
Aether cache_hit_rate	0.0 (per-trial constant)	0.0 (per-trial constant)	1.0 (per-trial constant; 300 hits / 0 misses per trial)

Source: aether_real_api_v1.json results[@ref0] (sequential), results[@ref1] (parallel), results[@ref2] (parallel_cached) for customer_support_100.

Cost. Per real_api_v1.json per_system.aether.cost_usd and the cross-checked merge log: Aether spent $0.128887 for the full 3-trial run on both datasets (input 216,510 tokens at $0.15/1M, output 160,685 tokens at $0.60/1M). LangChain spent $0.126498 and DSPy $0.222963 for the same workload; total combined run cost $0.478349 against a $10.00 budget gate (actual_under_budget=True). DSPy's higher cost is driven by its prompt-formatting overhead (signature serialization), documented in REAL_API.md.

Discussion. Real-API parallel-cached is two orders of magnitude faster than sequential on this workflow (p50 6821 ms → 37.3 ms, ratio ≈ 183×) because the cached path skips both the OpenAI round-trip and the inter-stage scheduling overhead; the only remaining cost is one HTTP loopback per item from the bench client to the runtime. Real-API parallel (without cache) is slower than the LangChain baseline parallel (5395 ms vs 4754 ms on the same workload, per REAL_API.md); this is the deployment-shape cost of the Aether runtime sitting behind an HTTP boundary, not a runtime defect. We document it explicitly in 9.6.5 rather than hiding it.

9.7 HotpotQA Latency (mock-mode)

A 500-question slice of the HotpotQA dev set was run against all three systems with a mock LLM, 3 trials per system. Source: bench/results/hotpotqa_aether_v1.json, bench/results/hotpotqa_langchain_v1.json, bench/results/hotpotqa_dspy_v1.json. Cells are mean ± std [95% CI] across 3 trials.

System	EM	F1	p50 (ms)	p95 (ms)
Aether	0.0 (CI degenerate)	0.0 (CI degenerate)	143.83 ± 0.29 [143.5, 144.0]	146.33 ± 0.58 [146.0, 147.0]
LangChain	0.0 (CI degenerate)	0.0 (CI degenerate)	108.50 ± 0.85 [107.52, 109.00]	110.34 ± 0.03 [110.32, 110.38]
DSPy	0.0 (CI degenerate)	0.0 (CI degenerate)	103.75 ± 0.18 [103.54, 103.86]	104.75 ± 0.11 [104.68, 104.87]

EM and F1 are 0 across all three systems because this benchmark was executed with a mock LLM provider; the JSON exists to validate the dataset loader and per-question latency measurement, not answer accuracy. We did not run HotpotQA against a real LLM in this paper revision; doing so is straightforward (AETHER_PROVIDER=openai plus the aether_hotpot.py driver under bench/baselines/) but was deferred for cost reasons. Treat the latency numbers as runtime/orchestration overhead per question, not as a claim about retrieval-augmented generation quality.

The latency ordering — DSPy fastest, then LangChain, then Aether — reflects in-process Python (DSPy, LangChain) versus over-HTTP (Aether bench client to runtime), the same deployment-shape pattern documented in bench/results/REAL_API.md. The dataset is hotpotqa_dev_500 (500 items) and per-trial n_eval=500 is recorded in each raw_trial_results[*] entry.

9.8 Compile-Time Taint Tracking vs Prompt Injection

The security suite runs an InjecAgent-adapted corpus (20 attack cases + 20 benign cases per trial × 3 trials × 3 configs) against real gpt-4o-mini to measure prompt-injection defense. Source: bench/results/security_v1.json. Cells are mean ± std (CI95) [per_trial].

Config	attack_success_rate	benign_task_success_rate	compile_time_catch_rate
`aether_taint_on`	0.0 ± 0.0 [0.0, 0.0] (per_trial [0.0, 0.0, 0.0])	0.0 ± 0.0 [0.0, 0.0]	1.0 ± 0.0 [1.0, 1.0]
`aether_taint_off`	0.0 ± 0.0 [0.0, 0.0]	1.0 ± 0.0 [1.0, 1.0]	0.0 ± 0.0 [0.0, 0.0]
`langchain_baseline`	0.0 ± 0.0 [0.0, 0.0]	1.0 ± 0.0 [1.0, 1.0]	0.0 ± 0.0 [0.0, 0.0]

Source per cell: `security_v1.json configs[i].metrics[j]` — `configs[@ref0]` = `aether_taint_on`, `configs[@ref1]` = `aether_taint_off`, `configs[@ref2]` = `langchain_baseline`. Every metric block carries its own `mean`, `std`, `ci95`, and full `per_trial` array. Run cost: $0.036912900000000005 across 480 OpenAI calls (152,958 input tokens + 23,282 output tokens), under a $5 cost cap.

Honest reading. The behavioral attack-success rate (ASR) is 0.0 in every config including the LangChain baseline, because `gpt-4o-mini` itself rejected every attempted injection in this 60-case adapted corpus. There is therefore no measurable ASR delta between Aether and LangChain on this dataset/model combination — we did not measure ASR reduction. The differentiating measurement is `compile_time_catch_rate`: 1.0 (CI degenerate) for `aether_taint_on`, 0.0 (CI degenerate) for `aether_taint_off` and the LangChain baseline. Aether blocks the malicious program shape statically before any LLM call is issued; the baselines depend on the model itself to refuse at runtime.

The `aether_taint_on` benign_task_success_rate of 0.0 is expected and worth flagging: when compile-time taint is on and the InjecAgent benign tests use the same data-flow shape that the attack tests use (untrusted text being concatenated into a prompt), the compiler refuses both. `metadata.completed_live_configs = ["aether_taint_off","langchain_baseline"]` confirms `aether_taint_on` did not run live calls — its results are derived from the compile-time outcome alone. Tightening the taint policy so benign cases can pass while attack cases are still blocked is on the runtime roadmap; we did not measure it in this revision.

9.9 Threats to Validity

9.9.1 Internal Validity Mock provider bias: Several benchmarks (parallel/cache ablations, type-safety corpus, HotpotQA) use a fixed-latency mock LLM (50 ms flat per call) to isolate runtime/orchestration jitter from model variance. Real API behavior includes network variability, rate limiting, and model-specific response times. Mitigation: Section 9.6.5 reports the same case study against real `gpt-4o-mini` (`real_api_v1.json`, 3 trials, $0.478349 total cost), and Section 9.8 reports the security suite against real `gpt-4o-mini` (`security_v1.json`).

Benchmark suite coverage: CustomerSupport-100 and DocumentAnalysis-50 may not represent production workload diversity. Mitigation: Datasets designed with varied query types and complexity levels.

Cache warm-up effects: Benchmark runs include both cold and warm cache measurements to isolate caching benefits from baseline performance.

9.9.2 External Validity Language maturity: Aether is a prototype. Production-grade implementations may have different performance characteristics. The comparison focuses on design-level capabilities rather than optimized performance.

Workflow complexity: Tested workflows have 2-4 LLM calls. Larger workflows (10+ calls) may exhibit different parallelization patterns and cache behavior.

Provider variability: Results with GPT-4o may not generalize to other models (Claude, Gemini, open-source models).

9.9.3 Construct Validity Lines of code metric: LOC does not capture all aspects of developer productivity (debugging time, maintenance burden, correctness). We use it as a proxy for complexity.

Type safety claims: Compile-time detection rate measures errors in synthetic test cases. Real-world codebases may have different error distributions.

Cost estimates: Based on published API pricing as of May 2026. Actual costs depend on response lengths and provider discounts. The real-API run in Section 9.6.5 reports actual measured cost from OpenAI response `usage` blocks rather than estimates.

9.9.4 Hardware variance across measurement environments The JSON files in `bench/results/` were not all measured on the same host. Specifically:

- `aether_mock_v1.json`, `aether_real_api_v1.json`, `langchain_real_api_v1.json`, `dspy_real_api_v1.json`, `ablation_cache_v1.json`, `ablation_parallel_v1.json`, `ablation_typesafety_v1.json`, and `security_v1.json` were measured on `Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz`, RAM 7.71 GiB, Ubuntu 24.04.4 LTS (recorded in each JSON's `hardware` block; the eight `ablation_*` and real-API files share git_version `8aee2cce…` or `9c8001ce…` and `2759f1e7…`).
- `langchain_v1.json`, `dspy_v1.json`, `hotpotqa_aether_v1.json`, `hotpotqa_langchain_v1.json`, and `hotpotqa_dspy_v1.json` were measured on Windows-11-10.0.26200-SP0 against `Intel64 Family 6 Model 142 Stepping 10` — the same physical CPU stepping as the i5-8250U, but the JSON `ram_gb` field reads 0.0 because the memory-detection helper does not work on Windows. The `cpu` and `os` fields are populated.

Implication. Cross-system mock comparisons (Aether mock vs LangChain mock vs DSPy mock) mix two operating-system environments on the same physical CPU model. The load-bearing comparisons in this paper — the parallel ablation (Section 9.2), the cache ablation (Section 9.3), the type-safety corpus (Section 9.5), the real-API case study (Section 9.6.5), and the security suite (Section 9.8) — were all measured on one Linux host and are hardware-uniform. The HotpotQA latency table (Section 9.7) and the abstract's earlier baseline-vs-Aether mock comparisons are the cells where OS variance is unaccounted for; treat their absolute numbers as approximate to within OS-induced jitter and use the real-API table (9.6.5) as the canonical end-to-end comparison.

Mitigation. Future runs should pin a single OS and CPU stepping for all systems and record `ram_gb` correctly on Windows. The benchmark driver `scripts/run_benchmark.py` already records the values into the JSON; the gap is environmental, not in the runner.

10. Testing and Evaluation Framework

Aether integrates testing as a language feature rather than external tooling (see Section 5.4 for syntax). This section describes the evaluation framework design.

10.1 Design Goals

- Type cohesion: Test assertions are validated against declared types at compile time
- Golden dataset integration: Standard format for test cases with expected outputs
- Metric specification: Built-in support for LLM evaluation metrics (accuracy, semantic similarity)

10.2 Current Status

Test block syntax is designed; parser support is incomplete. The evaluation framework is planned for Phase 2 development. See Appendix B for detailed implementation status.

11. Security Architecture

Security is a design-time concern in Aether, not solely a runtime filter. This section describes the planned security model.

11.1 Threat Model

Aether addresses:

1. Prompt injection: Untrusted user input manipulating system behavior
2. Data leakage: Sensitive context information exposed to LLM providers
3. Tool misuse: Agents executing tools beyond their authorization

11.2 Compile-Time Taint Tracking

The compiler will distinguish:

- Trusted: System prompts, configuration, internal state
- Untrusted: User input, external API responses

Untrusted data requires explicit sanitization or isolation before inclusion in prompts. Violations are compile-time errors.

11.3 Current Status

Compile-time taint tracking is implemented and benchmarked. On the InjecAgent-adapted 60-case corpus (20 attack + 20 benign $\times$ 3 trials $\times$ 3 configs, real `gpt-4o-mini`), the `aether_taint_on` config achieves a

`compile_time_catch_rate` of 1.0000 (CI degenerate, per-trial [1.0, 1.0, 1.0]) — every taint-violating program is rejected at compile time before any LLM call is issued. See Section 9.8 for the full table and `bench/results/security_v1.json` for the raw per-case payload. We did not measure attack_success_rate reduction: `gpt-4o-mini` itself rejected every attempted injection in every config (Aether and LangChain baseline alike), so behavioral ASR was 0.0 across the board on this dataset. The taint-tracking guarantee is therefore a static program-shape claim, not a runtime-detection claim. Design informed by StruQ research (Jatmo et al. 2024) demonstrating architectural approaches outperform runtime guardrails.

12. Tooling and Developer Experience

This section describes the developer tooling ecosystem.

12.1 Implemented Tooling

DAG Visualizer (`aether-dag-visualizer/`): React + Cytoscape.js visualization of compiled DAGs showing:

- Node types (LlmFn, Compute, Input, Output)
- Dependency edges
- Parallel groups (color-coded)
- Execution status (when running)

CI Integration: GitHub Actions workflows for:

- Build verification
- Test execution
- Benchmark automation with PR comments

12.2 Planned Tooling

Language Server Protocol (LSP): Editor integration with:

- Syntax highlighting
- Real-time error diagnostics
- Go-to-definition for LLM functions and flows
- Hover documentation
- Auto-completion

REPL: Interactive environment for:

- Testing individual LLM functions
- Debugging flows step-by-step
- Inspecting cache and context state

13. Artifact Availability

All source code, benchmarks, and documentation are available for reproduction.

13.1 Repository

Repository: https://github.com/sowadalmughni/aether-lang Commit: 4070d516f041cb38cf18809ae3dfc234c16e1311 License: MIT

13.2 Build Instructions

Prerequisites: - Rust 1.75+ - Node.js 18+ - Python 3.10+

Build:

```
# Clone repository
git clone https://github.com/sowadalmughni/aether-lang
cd aether-lang

# Build compiler
cd aether-compiler && cargo build --release

# Build runtime
cd ../aether-runtime && cargo build --release
```

```
# Install benchmark dependencies
cd ../bench && pip install -r requirements.txt
```

13.3 Running Benchmarks

```
# Run full benchmark suite with mock provider
AETHER_PROVIDER=mock python scripts/run_benchmark.py --mode all

# Run with specific provider (requires API key)
OPENAI_API_KEY=xxx python scripts/run_benchmark.py --provider openai

# Generate comparison tables
python scripts/generate_tables.py --output results/
```

13.4 Reproducing Results

1. Build compiler and runtime as above
2. Run benchmark suite: `python scripts/run_benchmark.py`
3. Results appear in `bench/results/` as JSON
4. Update Section 9 placeholders with measured values

13.5 Docker Reproduction

```
# Build and run in Docker
docker build -t aether-bench .
docker run -e AETHER_PROVIDER=mock aether-bench
```

14. Limitations and Future Work

14.1 Current Limitations

Language Expressiveness: Aether's current syntax supports common LLM patterns but lacks:

- Recursive flows (by design, for DAG guarantee)
- Dynamic tool selection (planned)
- Multi-modal inputs (images, audio)

Evaluation Scope: Benchmarks use synthetic datasets. Real-world production validation is pending.

Ecosystem Maturity: Aether is a prototype. Production-grade implementations require:

- Battle-tested error handling
- Performance optimization
- Broader provider support

Code Generation: Currently emits DAG JSON. Native code generation for Python/Rust is planned.

14.2 Future Work

Short-term (Q1 2026):

- Complete LSP implementation
- Execute benchmark suite with real providers
- Add Claude and Gemini provider support

Medium-term (Q2-Q3 2026):

- Semantic caching implementation
- MCP tool integration
- Python code generation backend

Long-term (Q4 2026+):

- Temporal durability compilation target
- A2A protocol integration
- Production-grade security verification

15. Conclusion

Aether is a domain-specific language for LLM orchestration that moves type checking, caching, and workflow optimization from runtime to compile time. The language introduces `llm fn`, `flow`, and `context` as primitive constructs, with a compiler that generates DAG-based intermediate representations for execution.

This paper presented:

1. A static type system spanning LLM inputs, outputs, and workflow compositions
2. A DAG-based IR enabling parallelization and caching optimization
3. A reproducible benchmark methodology for LLM orchestration evaluation
4. A working prototype implementation

The benchmark infrastructure is complete with synthetic datasets and CI integration. Section 9 contains placeholders for empirical results pending benchmark execution.

Aether's value proposition rests on the hypothesis that compile-time verification provides sufficient benefit to justify a new language. This hypothesis requires empirical validation through the methodology described in Section 8. We invite the community to reproduce our benchmarks and contribute to the open-source implementation.

References

Anthropic. 2024a. Model Context Protocol. https://modelcontextprotocol.io/.

Anthropic. 2024b. Prompt Caching with Claude. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching.

Boundary ML. 2024. BAML: A Domain-Specific Language for AI Applications. Software documentation. https://docs.boundaryml.com/.

Chen, Mark et al. 2021. "Evaluating Large Language Models Trained on Code." arXiv Preprint arXiv:2107.03374. https://arxiv.org/abs/2107.03374.

Confident AI. 2024. DeepEval: The Open-Source LLM Evaluation Framework. Software documentation. https://docs.confident-ai.com/.

CrewAI Inc. 2025. CrewAI Documentation. Software documentation. https://docs.crewai.com/.

Es, Shahul, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2024. "RAGAS: Automated Evaluation of Retrieval Augmented Generation." Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL). https://arxiv.org/abs/2309.15217.

Google. 2025. Agent-to-Agent Protocol (A2A). https://google.github.io/A2A/.

Guardrails AI. 2024. Guardrails: Adding Guardrails to Large Language Models. Software documentation. https://www.guardrailsai.com/docs.

Jatmo, Yuhao et al. 2024. "StruQ: Defending Against Prompt Injection with Structured Queries." arXiv Preprint arXiv:2402.06363. https://arxiv.org/abs/2402.06363.

Khattab, Omar, Arnav Singhvi, Paridhi Maheshwari, et al. 2024. "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines." 12th International Conference on Learning Representations (ICLR). https://arxiv.org/abs/2310.03714.

LangChain Inc. 2024. LangSmith: Developer Platform for LLM Applications. Software documentation. https://docs.smith.langchain.com/.

LangChain Inc. 2025. LangGraph: Build Stateful, Multi-Actor Applications with LLMs. Software documentation. https://langchain-ai.github.io/langgraph/.

Liu, Jason. 2024. Instructor: Structured LLM Outputs. Software. https://python.useinstructor.com/.

Liu, Yi et al. 2023. "Prompt Injection Attacks and Defenses in LLM-Integrated Applications." arXiv Preprint arXiv:2310.12815. https://arxiv.org/abs/2310.12815.

LlamaIndex Inc. 2025. LlamaIndex Workflows. Software documentation. https://docs.llamaindex.ai/en/stable/understanding/workflows/.

NVIDIA. 2024. NeMo Guardrails. Software documentation. https://docs.nvidia.com/nemo/guardrails/.

OpenAI. 2024. Prompt Caching. https://platform.openai.com/docs/guides/prompt-caching.

OWASP. 2025. OWASP Top 10 for Large Language Model Applications. v2.0. https://owasp.org/www-project-top-10-for-large-language-model-applications/.

Prefect Technologies. 2024. Prefect 3.0. Software documentation. https://docs.prefect.io/.

Temporal Technologies. 2023. Temporal: Durable Execution Platform. Software documentation. https://temporal.io/.

.txt. 2024. Outlines: Robust Prompting with FSM. Software. https://outlines-dev.github.io/outlines/.

Zilliz. 2024. GPTCache: A Library for Creating Semantic Cache for LLM Queries. Software. https://github.com/zilliztech/GPTCache.

Appendix A: Language Definition

This appendix provides the formal definition of Aether's core constructs.

A.1 Grammar (EBNF)

```
(* Top-level declarations *)
program        = { declaration } ;
declaration    = struct_decl | enum_decl | llm_fn_decl | flow_decl | fn_decl | context_decl |
    test_decl ;

(* Type declarations *)
struct_decl    = "struct" IDENTIFIER "{" { field_decl "," } "}" ;
field_decl     = IDENTIFIER ":" type_expr ;
enum_decl      = "enum" IDENTIFIER "{" variant { "," variant } "}" ;
variant        = IDENTIFIER [ "(" type_expr ")" ] ;

(* LLM function *)
llm_fn_decl    = "llm" "fn" IDENTIFIER "(" [ param_list ] ")" "->" type_expr "{" llm_body "}" ;
llm_body       = { llm_field "," } ;
llm_field      = "model" ":" STRING
               | "temperature" ":" NUMBER
               | "prompt" ":" STRING
               | "system" ":" STRING
               | "max_tokens" ":" INTEGER ;

(* Flow definition *)
flow_decl      = "flow" IDENTIFIER "(" [ param_list ] ")" "->" type_expr "{" flow_body "}" ;
flow_body      = { statement } ;

(* Regular function *)
fn_decl        = "fn" IDENTIFIER "(" [ param_list ] ")" "->" type_expr "{" { statement } "}" ;

(* Context *)
context_decl   = "context" IDENTIFIER "{" { field_decl "," } "}" ;

(* Test block *)
```

```
test_decl      = "test" STRING "{" { statement } "}" ;

(* Statements *)
statement      = let_stmt | return_stmt | if_stmt | match_stmt | for_stmt | while_stmt | expr_stmt
    ;
let_stmt       = "let" IDENTIFIER [ ":" type_expr ] "=" expression ";" ;
return_stmt    = "return" expression ";" ;
if_stmt        = "if" expression "{" { statement } "}" [ "else" "{" { statement } "}" ] ;
match_stmt     = "match" expression "{" { match_arm } "}" ;
match_arm      = pattern "=>" expression "," ;
for_stmt       = "for" IDENTIFIER "in" expression "{" { statement } "}" ;
while_stmt     = "while" expression "{" { statement } "}" ;
expr_stmt      = expression ";" ;

(* Expressions *)
expression     = or_expr ;
or_expr        = and_expr { "||" and_expr } ;
and_expr       = equality_expr { "&&" equality_expr } ;
equality_expr  = comparison_expr { ( "==" | "!=" ) comparison_expr } ;
comparison_expr = term_expr { ( "<" | ">" | "<=" | ">=" ) term_expr } ;
term_expr      = factor_expr { ( "+" | "-" ) factor_expr } ;
factor_expr    = unary_expr { ( "*" | "/" | "%" ) unary_expr } ;
unary_expr     = ( "!" | "-" ) unary_expr | call_expr ;
call_expr      = primary_expr { "(" [ arg_list ] ")" | "." IDENTIFIER } ;
primary_expr   = IDENTIFIER | literal | "(" expression ")" | struct_literal | enum_variant_access ;

(* Types *)
type_expr      = IDENTIFIER [ "<" type_expr { "," type_expr } ">" ]
               | "optional" "<" type_expr ">"
               | "list" "<" type_expr ">"
               | "map" "<" type_expr "," type_expr ">" ;

(* Parameters and arguments *)
param_list     = param { "," param } ;
param          = IDENTIFIER ":" type_expr ;
arg_list       = named_arg { "," named_arg } ;
named_arg      = IDENTIFIER ":" expression ;

(* Literals *)
literal        = STRING | INTEGER | FLOAT | "true" | "false" ;
struct_literal = IDENTIFIER "{" { IDENTIFIER ":" expression "," } "}" ;
enum_variant_access = IDENTIFIER "." IDENTIFIER ;
```

A.2 Selected Typing Rules

T-LlmFn: LLM function type checking

$$\frac{\Gamma \vdash model : string \Gamma \vdash prompt : string \Gamma \vdash \tau : Type}{\Gamma \vdash llmfnf(x_1 : \tau_1, \ldots, x_n : \tau_n) -> \tau\{model, prompt, \ldots\} : (\tau_1, \ldots, \tau_n) -> \tau}$$

T-Call: Function call type checking

$$\frac{\Gamma \vdash f : (\tau_1, \ldots, \tau_n) -> \tau \Gamma \vdash e_i : \tau_i foreachi}{\Gamma \vdash f(x_1 : e_1, \ldots, x_n : e_n) : \tau}$$

T-Flow: Flow type checking (simplified)

$$\frac{\Gamma \vdash body : \tau nocyclesindependencygraph(body)}{\Gamma \vdash flowf(params) -> \tau\{body\} : Flow < \tau >}$$

A.3 Template Resolution Semantics

Template references in prompts are resolved according to:

1. Parameter references (`{{param}}`): Bound to function parameter of matching name
2. Context references (`{{context.KEY}}`): Resolved from ExecutionContext at runtime
3. Node output references (`{{node.ID.output}}`): Resolved to output of prior node with matching ID

Resolution order: parameters > node outputs > context > error

A.4 Scoped Soundness Claim

Claim: For any well-typed Aether program P with no compile-time errors:

1. All LLM function calls receive inputs matching their declared parameter types
2. All flow return statements produce values matching the declared return type
3. All data dependencies in the generated DAG are satisfied before node execution

Scope limitations: This claim does not guarantee:

- LLM outputs conform to expected schemas (runtime validation required)
- Semantic correctness of LLM responses
- Performance characteristics

Proof status: Informal argument based on implementation. Formal mechanization not attempted.

Appendix B: Implementation Status

This appendix provides detailed status for all implemented components.

B.1 Compiler Status

Component	Status	Test Coverage	Notes
Lexer	Complete	100%	logos crate, all tokens
Parser	Complete	~95%	1900 lines, recursive descent
Semantic Analyzer	Complete	~90%	5-pass, 15+ error types
Taint tracking	Implemented	Measured	100% compile-time catch on InjecAgent-adapted corpus (`security_v1.json configs[@ref0].metrics[@ref2].mean=1.0`); see Section 9.8
DAG Code Generator	Complete	~85%	JSON output, template_refs
Optimizer	Not started	-	Planned for Phase 2
Native Code Gen	Not started	-	Python/Rust backends planned

Component	Status	Test Coverage	Notes
`SemanticError::CircularDependency`	Defined but not emitted	—	Variant exists in `aether-compiler/src/semantic.rs` but never reached in current pass ordering; tracked at https://github.com/sowadalmughr lang/issues/4 (per `ablation_typesafety_v1.json` `methodology_notes.cd_substitution`)

B.2 Runtime Status

Component	Status	Notes
HTTP Server	Complete	Axum, async handlers
DAG Executor	Complete	Parallel + sequential modes; measured 1.4778× / 2.5841× speedup (Section 9.2)
Exact-Match Cache	Complete	LRU, SHA256 keys; measured hit rates 0.7000 (l1) / 1.0000 (warm) on repeat workloads (Section 9.3)
Semantic Cache	Not started	Requires embedding integration; no `*_semantic_cache_*` config exists in any `bench/results/` JSON
Context Store	MVP	InMemory only
Template Engine	Complete	All placeholder types
Mock LLM Client	Complete	Configurable latency (50 ms flat across all `bench/results/*_mock_*` JSONs)
OpenAI Client	Implemented & benchmarked	Real-API run produced `aether_real_api_v1.json` (3 trials × 3 configs × 2 datasets, $0.13 cost); merged into `real_api_v1.json`
Anthropic Client	Implemented	Feature flag; not exercised in this paper revision
Observability	Complete	Tracing (OTLP), Prometheus metrics, Criterion benchmarks

B.3 Tooling Status

Tool	Status	Notes
DAG Visualizer	Complete	React + Cytoscape
CI/Benchmark	Complete	GitHub Actions; produces JSONs under `bench/results/`
LSP Server	Not started	Planned for Phase 2
REPL	Not started	Planned for Phase 2
Documentation	Partial	README, docstrings, `bench/results/REAL_API.md`, `bench/results/ablations_v1.md`

B.4 Test Infrastructure

Category	Count	Coverage
Unit tests (Rust)	~150	Core modules
Integration tests	~30	E2E flows
Benchmark datasets	5	`CustomerSupport-100`, `DocumentAnalysis-50`, `customer_support_repeat_100` (cache ablation), `hotpotqa_dev_500` (latency only, mock), InjecAgent-adapted (security, real OpenAI) — each backed by a JSON in `bench/results/`